

Task Split & Checklist

分工 (division of labor) for two people, from where the codebase stands today to a Google Play submission. Each item below is concrete enough to check off — no "figure it out later" steps.

DEADLINE	TARGET STORE
Sep 30, 2026 · 11:45pm PT	Google Play only
HARD REQUIREMENT	CURRENT STATE
RevenueCat SDK wired	Mock mode, no Android platform, no RevenueCat

OVERVIEW

What Sports Buddy is

Product summary, for anyone new to the project

Sports Buddy is a mobile-first social app that finds you a real person to play sport with. Instead of a generic "nearby" list, it ranks candidates with a deterministic, rule-based compatibility score (shared sport, skill level, overlapping availability, area, budget — no AI, no randomness), and instead of stopping at "here's a match" it carries two people all the way through connecting, chatting, and collaboratively agreeing on a sport, a time, a budget and a venue, down to a confirmed, calendar-exportable session.

It is built React 19 + TypeScript + Vite + Tailwind v4, packaged for mobile with Capacitor, on a Firebase backend (Auth + Firestore) — currently still running against a local mock/localStorage backend, with the Firebase paths written and tested but never pointed at a real project. 13 of the project's own 20 planned build steps are shipped; the remaining gap to a Google Play submission is Platform & Monetization work (Player A) and Product & Safety work (Player B), detailed from page 3 onward.

USER FLOW

The journey, start to finish

Every screen a new user passes through, in order

- 1 Sign up / sign in** </auth/register> · </auth/login>
Email + password, or Google (web popup — the one Capacitor risk noted on page 4).
- 2 Onboarding — 8 steps** </onboarding>
Sports played, skill per sport, intent, availability, area + travel radius, budget, short bio, preview. Saved to Firestore once, on completion.
- 3 Home** </home>
Snapshot of the soonest upcoming activity and entry points into the rest of the app.
- 4 Discover** </discover>
Ranked buddy feed — every candidate scored live against the signed-in user, with filters (sport, skill, area, budget).
- 5 View a buddy's profile** </discover/:userId>
Discovery-safe profile plus the full compatibility breakdown, and Connect.
- 6 Connect → mutual connection**
A pending request becomes Connected the moment the other person connects back — live, no refresh.
- 7 Chat** </messages/:conversationId>
Real-time 1:1 messaging, only ever available between connected pairs.
- 8 Plan Together**
From the chat: collaboratively propose and agree a shared sport, a time slot, and a budget — both people must accept each one.
- 9 Choose a venue**
Google Places search centred on the midpoint of both people's areas (never anyone's actual location).
- 10 Confirm the activity**
Either participant confirms once sport, time, budget and venue are all agreed — turns the plan into an immutable Activity.
- 11 Activities — upcoming & past** </activities>
Month-grouped history, and a one-tap "Add to calendar" (.ics) export.
- 12 Profile & Settings** </profile> · </settings>
Available throughout — edit the profile, adjust discovery/privacy/notification preferences.

What's already built

13 of 20 planned steps, all with passing tests

Account & profile

Authentication	Email/password and Google sign-in, idempotent user document creation.
Onboarding	8-step guided setup, resumable, written once on completion.
Profile & preferences	Full profile editing, discovery/privacy/notification preferences, profile-strength indicator.

Discovery & matching

Discover feed	Ranked candidates with filters; excludes yourself and non-discoverable profiles.
Compatibility engine	Pure, deterministic 5-factor weighted score — sport 35 / skill 20 / availability 20 / area 15 / budget 10 — with human-readable reasons.

Social & planning

Connect / mutual connections	One transactional pair document, live perspective (pending / connected) per viewer.
Realtime chat	Paginated (20/page), atomic send + preview update, scoped realtime listeners only.
Plan Together	Collaborative sport / time / budget proposals with versioned agreement, no stale-accept races.
Venue discovery	Google Places (New) search around the midpoint of two areas; 12-venue mock dataset by default.
Confirmed activities	Immutable snapshot on confirmation; either participant may confirm.
History & calendar	Upcoming/past tabs (month-grouped), derived — never stored — temporal state, .ics export.

Platform

Responsive shell	Dedicated phone / tablet / desktop layouts — bottom nav, rail, sidebar — not one squeezed design.
Design system	Full token-based theming, dark-mode-first (details on the next page).
Security hardening	Input validation, XSS/injection audit, Firestore rules for every collection (97 emulator tests).

What's not built yet

Required before Google Play / Shipaton submission

BLOCKING

RevenueCat + monetization	No SDK, no paid feature exists. See Player A, Phase 2.
Android platform + real Firebase	No native project generated, no live backend ever run. See Player A, Phase 1.
Block / report	Required by Google Play's User-Generated-Content policy for stranger messaging. See Player B, Phase 1.
Account deletion	Flagged by the project itself as required before production. See Player B, Phase 2.
Store listing (privacy policy, icon, screenshots, data safety)	None exist yet. See Player B, Phase 3.

Deliberately deferred — not required for this submission

BY DESIGN

Activity ratings, attendance, "completed" state	Past only means the end time passed — no review or reliability system exists or is planned yet.
Reschedule / cancel a confirmed plan	Plans and activities are intentionally immutable once agreed / confirmed.
Disconnect / unfriend (beyond block)	A connected relationship can be blocked but not otherwise dissolved.
Read receipts, typing indicators, presence	Explicitly not implemented — a fake one was judged worse than none.
Media messages (photos, etc.) in chat	Text only.
Photo upload	Needs Firebase Storage; initials avatar is the fallback today.
Native calendar plugin, venue booking/pricing, travel-time / ETA	.ics export covers calendar; venues stay list + map only, no booking or live pricing.
Native Google sign-in plugin, analytics/crash dashboards beyond the Shipaton perks	Deferred to a dedicated native phase.

Q: 除了这些，其他功能不需要做/修改吗？ — is there other feature work needed?

No. Everything in the "deliberately deferred" table above is an intentional scope cut already documented in the project's own build plan, not a gap that blocks Shipaton or Google Play — leave all of it alone. The complete list of what actually blocks submission is the five rows in the "blocking" table above, which map one-to-one onto Player A's and Player B's phase checklists on the following pages. Two small items worth folding into Player A's RevenueCat work specifically (added to that checklist): a **Restore Purchases** control, and a short **Terms of Service** page covering the subscription, alongside the privacy policy.

Theme

Dark-mode-first, one token file, no other raw colour anywhere in the app

Brand personality, from the project's own design-system doc: *"Energetic but premium. Dark, confident surfaces with one loud fluorescent lime accent, sport orange for momentum, and a restrained warm gold for premium moments."* Reference points are Apple (spacing, restraint, soft depth) and Strava (metric-forward, unmistakable primary CTA) — inspiration only, nothing cloned.

LIGHT THEME

 **background**
#f6f7f8

 **card**
#ffffff

 **primary**
#46700d

 **secondary**
#fc5a1a

 **accent (gold)**
#7a5b00

 **success**
#167c4a

 **warning**
#b45309

 **destructive**
#cc2a2e

DARK THEME (PRIMARY IDENTITY)

 **background**
#0b0c10

 **card**
#17191f

 **primary (lime)**
#c4ff3d

 **secondary**
#fc5a1a

 **accent (gold)**
#e7c768

 **success**
#3fd37a

 **warning**
#ffb020

 **destructive**
#ff5a5f

Every one of these is a CSS variable in `src/styles/theme.css` — the single file allowed to hold a raw colour value. Components only ever consume the semantic name (`bg-primary` , `text-destructive` ...).

Type scale system font stack — no webfont loaded

ROLE	SIZE	WEIGHT	USED FOR
display	34px	700	Hero numbers, splash
heading-1	28px	700	Page titles (desktop)
heading-2	22px	600	Section headings
heading-3	18px	600	Card titles
title	16px	600	List item titles
body	15px	400	Default reading text
label / body-small	13px	600 / 400	Form labels, secondary text
caption	11px	600, +6% tracking	Uppercase eyebrows, badges
metric	26px	700, tabular	Compatibility score, stats

Layout & motion

BREAKPOINT	NAVIGATION	CHROME WIDTH
Phone < 768px	Bottom nav, fixed	—
Tablet 768–1023px	Navigation rail	80px
Desktop ≥ 1024px	Sidebar	240px

Radius scale 6 / 10 / 13 / 16 / 20 / 24 / 32 / 40px + full pill. One motion system app-wide: 120 / 200 / 320ms on a single decelerating easing curve — nothing hand-picks its own transition.

Platform & Monetization

Firebase → Android build → RevenueCat + Billing → Play Console

Phase 1 — Firebase + Android scaffolding (days 1–4)

- ☐ **Create a Firebase project** at `console.firebase.google.com`
Analytics can stay off, it isn't used.
- ☐ **Register a Web app**, copy the `firebaseConfig` values
- ☐ **Enable Email/Password sign-in** in Authentication
- ☐ **Enable the Google sign-in provider**, add every dev domain under Authorized domains
Note: this is the web popup flow. It's documented as unreliable in a Capacitor WebView — don't block launch on it working perfectly on Android.
- ☐ **Create the Firestore database** (production mode, a region close to your users, e.g. `asia-southeast1`)
- ☐ **Deploy the existing security rules:** `firebase login` → `firebase use --add` → `firebase deploy --only firestore:rules`
`firestore.rules` is already written and passes 97 emulator tests — this is config, not new code.
- ☐ **Switch the app to live Firebase:** `cp .env.example .env`, set `VITE_DATA_SOURCE=firebase`, fill in the Firebase keys
- ☐ **Run the two-user verification scripts** already written in `docs/firebase.md` §9b (connections) and §9c (chat)
This has never been run against a real project in this environment — it's the first real proof the live backend works.
- ☐ `npm run cap add android` to generate the native project
No android/ folder exists yet.
- ☐ **Set app icon + generate a release keystore**, wire signing config in Gradle
- ☐ `npm run cap:sync` and confirm the app boots on an emulator or device

Phase 2 — Monetization + RevenueCat (days 5–10)

- ☐ **Decide what's actually paid** — coordinate with Player B on product fit
e.g. a "Sports Buddy Plus" tier: unlimited daily connects, advanced discovery filters, priority ranking. The app currently has nothing to sell.
- ☐ **Create a RevenueCat account + project**
This is the Shipaton-mandatory step, and unlocks the next perk tier ("RevenueCat project created").
- ☐ **Create the subscription/product in Google Play Console**
Needs a Play Console account to exist first — sequence with Phase 4 if it slips earlier.
- ☐ **Install RevenueCat's Capacitor plugin**, initialize the SDK on app start
- ☐ **Build the purchase flow / paywall** (UI polish is Player B's, wiring is yours)
- ☐ **Complete one sandbox purchase** using a Play license tester account
Unlocks the "first test purchase" perk tier.
- ☐ **Add a Restore Purchases control**
Standard expectation for any subscription app — cheap to add while the purchase flow is already being built.
- ☐ **Write a short Terms of Service page** covering the subscription (price, renewal, cancellation), alongside Player B's privacy policy

Phase 3 — Build pipeline, crash tracking, push (days 8–12)

- ☐ **Claim the Codemagic perk** (500 build min/month), connect the repo, configure an Android build workflow that outputs a signed AAB
Skips needing local Android Studio/Gradle on Windows.
- ☐ **Claim the Sentry perk** (\$100 credits), wire basic error tracking
Crashlytics was only ever "planned" in this project, never wired — Sentry covers the same gap for free.
- ☐ **Optional — OneSignal push notifications** (100% off Growth plan, 3 months): project setup, Capacitor plugin, a single "you have a new message" notification
Matches the project's own next planned build step.

Phase 4 — Play Console submission (days 13–17)

- ☐ **Google Play Console developer account** (\$25 one-time)
- ☐ **Create the app listing, upload the signed AAB**
- ☐ **Fill the Data Safety form** — declare: email, approximate location (area only, no GPS), chat messages
Get the exact field list from Player B's privacy policy draft so both match.
- ☐ **Complete the content rating questionnaire**
- ☐ **Enroll in Play App Signing**
- ☐ **Release to Closed/Internal testing first**, confirm install works
- ☐ **Promote to Production** once review clears
Review typically takes a few business days — this is why Phase 4 starts with ~2 weeks of runway left, not 2 days.
- ☐ **Complete one real purchase** post-launch
Unlocks the final "first real purchase" perk tier.

Phase 5 — Buffer (days 18–19)

- ☐ **Confirm the Play Store listing is publicly installable**, not stuck in internal testing
- ☐ **Hand Player B the live Play Store link + RevenueCat project link** for the Devpost submission

Don't skip: RevenueCat SDK is a hard Shipaton submission requirement — an app with no monetization at all cannot be submitted.

PLAYER B

Product & Safety

Block/report → account deletion → store assets → visual QA → submission

Phase 1 — Block / report (days 1–4)

- ☐ **Design the data model:** extend the existing connection domain with a `blocked` state and a report reason enum, mirroring the STEP 8 pattern already in the codebase (pure function in `src/lib/connection.ts` → `connection-service.ts` → repository → Firestore rules)
- ☐ **Add a "Block" action** to the candidate profile and the chat header
- ☐ **Add a "Report" flow** (reason picker + optional note), written to a simple `reports` collection
No moderation dashboard needed for the MVP — just capture it.
- ☐ **Update Firestore rules + emulator tests** for the new collection(s)
- ☐ **Hide a blocked user** from Discover and from the Messages list

Phase 2 — Account deletion (days 3–6)

- ☐ **Add a Delete Account control** in Settings → Account, behind a confirmation dialog
- ☐ **Clear** `users/{uid}` and `publicProfiles/{uid}` on delete
- ☐ **Decide the simplest safe handling for connections/messages** (e.g. leave them referencing a removed id) rather than a full cascading delete
CLAUDE.md flags orphaned data as the reason this was skipped originally — a minimal, honest approach beats an elaborate one under this timeline.
- ☐ **Delete the Firebase Auth user last**, after Firestore cleanup succeeds

Phase 3 — Store assets + privacy policy (days 5–9)

- ☐ **App icon** (512×512) + Android adaptive icon layers
- ☐ **Feature graphic** (1024×500)
- ☐ **Capture phone/tablet/desktop screenshots**, dark + light, across Discover / Chat / Planning / Activities
The responsive layouts already exist for this — it's capture, not new design work.
- ☐ **Write the store listing copy** — short description, full description
- ☐ **Write and publish a Privacy Policy page** with a public URL (a static page is enough) covering: what's collected (email, approximate area, chat content), and that there's no GPS/precise location
- ☐ **Link the privacy policy** in Play Console and in the app's Settings screen

Phase 4 — Visual QA (days 8–14)

- ☐ **Real device (or DevTools device emulation) pass at 390 / 768 / 1024 / 1440px, dark + light**
Every single build step in this project's history notes this pass "was not automated in this environment" — it has never actually happened.
- ☐ **Walk every flow end-to-end** on Player A's Android build once it exists: onboarding → discover → connect → chat → plan → venue → confirm → calendar export
- ☐ **File anything broken back to Player A**, don't silently work around it

Phase 5 — Submission materials (days 14–18)

- ☐ **Record a short demo video** — check the current Devpost page for Shipaton's exact video requirements
- ☐ **Write the Devpost submission:** problem, solution, tech stack, how RevenueCat is used, screenshots, video, live Play Store link
- ☐ **Submit to Devpost** before the Sep 30 cutoff — aim for a few days early, not the last hour

Don't skip: block/report is a Google Play policy requirement for any app that lets strangers message each other, not just a nice-to-have — a stranger-chat app without it risks rejection or removal.

BOTH

Joint checklist

Run together, right before submitting

- ☐ **Install the signed build on a real Android device** and do one full, unhurried smoke test
 - ☐ **Confirm a RevenueCat purchase actually completes end-to-end on-device**, not just in a simulator
 - ☐ **If push notifications were built, confirm one actually arrives on-device**
 - ☐ **Confirm the Play Store listing is public and installable** by someone outside the testing list
 - ☐ **Submit to Devpost / Shipaton**
-

Sources: shipaton.com/faq (deadline, submission requirements), shipaton.com/ship-kit (perk details). Everything else is read from this repository's own CLAUDE.md, README.md and docs/firebase.md, and from the test/build run against it directly.